skraft

Fonctionnement détaillé du pipeline

Étapes, portes, lentilles, agents et code
Entrées et sorties, premier passage et rejeu
Erreurs bloquantes et non bloquantes

AgentStudio — extension `skraft` et workflow `workflows/skraft/`
État du code : 15 octobre 2026, commit suivant `37349d3`
Édition smartphone

Sommaire

1 Lire ce document 1
1.1 À quoi sert skraft 1
1.2 Deux implémentations, un comportement 1
1.3 Vocabulaire 2
1.4 Légende des organigrammes 3
2 Vue d'ensemble du pipeline 5
2.1 Les étapes, dans l'ordre 5
2.2 Le niveau de difficulté règle les moyens 6
2.3 Les agents 6
2.4 Les interventions humaines 8
3 Mécanismes partagés par toutes les étapes 10
3.1 Le checkpoint de phase 11
3.2 Le dossier de passation 12
3.3 Le tour d'un agent 12
3.4 La porte 13
3.5 La revue par lentilles 13
3.6 Le budget d'essais et le rejeu 19
3.7 Le commit et la traçabilité 20
4 Préparation : INIT, CLASSIFY et ROUTE 21
4.1 INIT : lire la story et vérifier le terrain 21
4.2 CLASSIFY : estimer la difficulté 22
4.3 ROUTE : trancher les questions ouvertes 23
5 RESEARCH : comprendre l'existant 25
5.1 Rôle 25
5.2 Entrées et sorties 25
5.3 Qui exécute quoi 26
5.4 La porte « citations » 26
5.5 Organigramme 27
5.6 Premier passage et rejeu 27
6 DESIGN : concevoir et décider 29
6.1 Rôle 29

6.2 Entrées et sorties 29
6.3 Qui exécute quoi 29
6.4 La porte « cohérence » 30
6.5 La ratification 32
6.6 Organigramme 33
6.7 Premier passage et rejeu 33
7 DISTILL : écrire les scénarios et les tests d'acceptance 35
7.1 Rôle 35
7.2 Entrées et sorties 35
7.3 Qui exécute quoi 36
7.4 Les quatre portes, dans l'ordre 36
7.5 Le gel des tests d'acceptance 38
7.6 Organigramme 39
7.7 Premier passage et rejeu 39
8 DELIVER : livrer en TDD, scénario par scénario ... 41
8.1 Rôle 41
8.2 Entrées et sorties 41
8.3 Qui exécute quoi 41
8.4 Planification 43
8.5 La boucle TDD d'un scénario 45
8.6 La porte de fin de scénario 46
8.7 Les portes de fin de DELIVER 47
8.8 Corrections ciblées et arbitrages 48
8.9 La revue de DELIVER 50
8.10 Premier passage et rejeu 50
9 Interactions entre les étapes 52
9.1 Ce qui ne passe pas d'une étape à l'autre 55
10 Erreurs : bloquantes et non bloquantes 56
10.1 Les catégories d'erreur 56
10.2 Les situations bloquantes 57
10.3 Les situations non bloquantes 58
10.4 Les gardes de sécurité 59

11 Extension et workflow : différences 61
11.1 Écarts relevés en préparant ce document . . 62
11.2 Sources 63

1 Lire ce document

1.1 À quoi sert skraft

skraft transforme une story (un besoin métier décrit avec ses critères d'acceptation) en code livré, testé et revu. Le travail est découpé en étapes. Dans chacune, des agents (des modèles de langage) produisent des fichiers, et le code d'AgentStudio vérifie ce qu'ils ont produit avant de laisser passer.

Le principe qui guide tout le document : **le LLM propose, le code décide et vérifie**. Un agent écrit ; il ne décide jamais seul qu'une étape est terminée. Les preuves (les exécutions de tests) sont produites par le code, jamais rapportées par un agent.

1.2 Deux implémentations, un comportement

skraft existe sous deux formes, qui doivent se comporter de la même façon :

- **l'extension** `extensions/skraft/` : une machine à états (un réducteur pur, plus des effets qui font les entrées-sorties) ;
- **le workflow** `workflows/skraft/` : un script JavaScript rejouable, qui appelle les mêmes agents et les mêmes portes.

Ce document décrit le comportement commun, à partir du code de l'extension. Les écarts du workflow sont rassemblés au Chapitre 11.

1.3 Vocabulaire

Étape
Définition : Un moment du pipeline. Les quatre étapes de production sont les **phases** RESEARCH, DESIGN, DISTILL et DELIVER. Avant elles, trois étapes de préparation : INIT, CLASSIFY et ROUTE.

Stade
Définition : L'état précis de la machine à l'intérieur d'une étape : SPECIALIST (un agent travaille), ARTIFACT_GATE (la porte s'exécute), REVIEW (les lentilles jugent), COMMIT, BLOCKED, ESCALATED, etc.

Agent
Définition : Un modèle de langage, lancé dans une **session neuve** avec un rôle (par exemple `skraft-dev`), un dossier de passation, des outils autorisés et un périmètre d'écriture. Il termine son tour par un **outil terminal** : `report_phase`, `submit_lens_verdict`, etc.

Porte (gate)
Définition : Un contrôle exécuté par le **code**, sans LLM, à la fin d'un tour d'agent. Il lit les fichiers, lance les tests, calcule des empreintes. Il rend « passe » ou « rejet » avec des motifs exacts.

Lentille
Définition : Un agent de **revue**, spécialisé sur un angle (lisibilité, intégrité des tests, architecture, cohérence avec les mesures). Il ne modifie rien : il rend un verdict et des défauts. Le verdict final est calculé par le code à partir de toutes les lentilles.

Essai
Définition : Un passage dans une phase. Chaque rejet (de porte ou de revue) en consomme un. Le budget est de 2 essais au niveau `simple`, 3 aux autres.

Rejeu
Définition : Tout nouveau passage après un rejet : relance ciblée (même session, avec le motif), reprise après une escalade, ou rejeu demandé par l'humain.

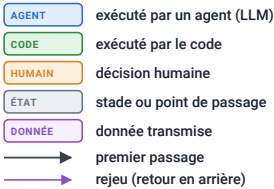
Escalade
Définition : La main passe à l'humain : le code a épuisé ses options, ou une panne l'empêche de continuer. Le dépôt est remis au point de départ de l'étape et les agents oublient leurs sessions.

Rembobinage
Définition : Retour du dépôt à un **checkpoint** (point de retour pris par le code), toujours accompagné de l'oubli des sessions des agents concernés.

Dérogation
Définition : Une exemption précise d'une porte (un mutant, une ligne non couverte), accordée après arbitrage et tracée.

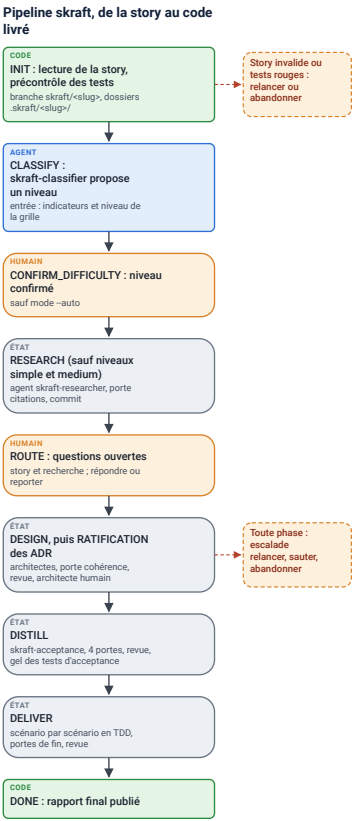
1.4 Légende des organigrammes

Chaque étape est illustrée par un organigramme vertical. La couleur dit **qui exécute** : un agent, le code ou un humain. Les flèches noires suivent le premier passage ; les flèches violettes montrent les retours en arrière (rejeu) ; les cadres en pointillés, à droite, montrent les sorties vers l'humain.


AGENT
exécuté par un agent (LLM)
CODE
exécuté par le code
HUMAIN
décision humaine
ÉTAT
stade ou point de passage
DONNÉE
donnée transmise
premier passage
rejeu (retour en arrière)

2 Vue d’ensemble du pipeline

2.1 Les étapes, dans l’ordre



L’ordre des phases est fixé par le code : RESEARCH, DESIGN, DISTILL, DELIVER. La phase suivante est toujours la première qui n’est ni faite ni sautée. RE-

SEARCH est sautée aux niveaux `simple` et `medium`. Avant toute phase autre que RESEARCH, les questions ouvertes doivent avoir une réponse (étape ROUTE), et les ADR proposés doivent être ratifiés : DISTILL est inatteignable tant qu'un ADR est au statut `Proposed`.

2.2 Le niveau de difficulté règle les moyens

Le niveau ne change jamais la logique de contrôle ; il change les moyens engagés.

simple
RESEARCH : sautée
Granularité de DELIVER : un tour par scénario
Lentilles : cold-reader, test-integrity
Essais par phase : 2

medium
RESEARCH : sautée
Granularité de DELIVER : un tour par scénario
Lentilles : les quatre
Essais par phase : 3

medium-hard
RESEARCH : exécutée
Granularité de DELIVER : un tour par scénario
Lentilles : les quatre
Essais par phase : 3

challenging
RESEARCH : exécutée
Granularité de DELIVER : sessions séparées par pas
Lentilles : les quatre
Essais par phase : 3

2.3 Les agents

`skraft-classifier`
Étape : CLASSIFY

Sortie : `submit_difficulty`
Outils : aucun
Écrit dans : rien

`skraft-researcher`
Étape : RESEARCH
Sortie : `report_phase`
Outils : `list_files`, `report_blocker`
Écrit dans : `.skraft/**`

`skraft-architect-model`
Étape : DESIGN
Sortie : `report_phase`
Outils : `list_files`, `report_blocker`
Écrit dans : `.skraft/**`

`skraft-architect-decide`
Étape : DESIGN
Sortie : `report_phase`
Outils : `list_files`, `render_adr`, `report_blocker`
Écrit dans : `.skraft/**` ; les ADR via `render_adr`

`skraft-acceptance`
Étape : DISTILL
Sortie : `report_phase`
Outils : `list_files`, `run_tests`, `report_blocker`
Écrit dans : `.skraft/**`, tests d'acceptance, sources

`skraft-test`
Étape : DELIVER
Sortie : `report_phase`
Outils : `list_files`, `run_tests`, `run_lint`, `report_blocker`
Écrit dans : tests unitaires

`skraft-dev`
Étape : DELIVER
Sortie : `report_phase`
Outils : `list_files`, `run_tests`, `run_lint`, `report_blocker`

Écrit dans : sources

`skraft-lens-*` **(4)**
Étape : revues
Sortie : `submit_lens_verdict`
Outils : `report_blocker`
Écrit dans : rien

`skraft-mutation-arbiter`
Étape : DELIVER
Sortie : `submit_lens_verdict`
Outils : aucun
Écrit dans : rien

Deux fichiers sont toujours en lecture seule pour tous les agents : la barre de qualité du projet (`.agent-studio/quality.yaml`, ou l'ancien `.skraft/quality.json`). Les ADR déjà décidés le sont aussi. Le shell est toujours refusé.

2.4 Les interventions humaines

L'humain n'est jamais attendu de façon bloquante dans un processus : chaque décision est une **demande d'interaction** persistée, à laquelle il répond quand il veut. Certaines demandes sont réservées à des profils.

Difficulté de US-N ?
Quand : CLASSIFY
Options : le niveau proposé, un cran au-dessus, un cran en dessous
Profils : tous

Question ouverte
Quand : ROUTE
Options : répondre (texte libre), reporter
Profils : po, développeur, architecte, administrateur

Question de l'agent
Quand : un agent bloqué
Options : les choix de l'agent, ou « Répondre »
Profils : po, développeur, architecte, administrateur

Question de la revue
Quand : une lentille bloquée
Options : les choix de la lentille
Profils : po, développeur, architecte, administrateur

Ratifier ADR-N
Quand : DESIGN
Options : accepter, rejeter, amender
Profils : architecte

Arbitrer un désaccord de revue
Quand : un défaut contesté deux fois, maintenu deux fois
Options : donner raison à l'agent, ou à la lentille
Profils : développeur, architecte, administrateur

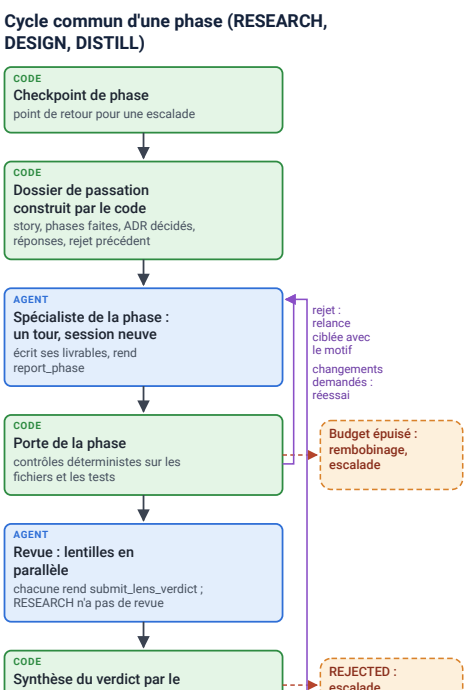
Arbitrer un mutant
Quand : DELIVER, mutant contesté
Options : dérogation, à corriger
Profils : développeur, architecte, administrateur

Arbitrer un défaut de couverture
Quand : fin de DELIVER, couverture contestée
Options : dérogation, à tester
Profils : développeur, architecte, administrateur

Pipeline skraft bloqué
Quand : escalade
Options : selon le cas : relancer, sauter, continuer sans commiter, abandonner
Profils : développeur, architecte, administrateur

3 Mécanismes partagés par toutes les étapes

Les phases RESEARCH, DESIGN, DISTILL et DELIVER suivent le même cycle. Ce chapitre décrit une fois pour toutes ce qu'elles ont en commun ; les chapitres suivants ne décrivent que ce qui est propre à chacune.



3.1 Le checkpoint de phase

À l'entrée de chaque phase, le code prend un **checkpoint** : une photographie de l'arbre de travail dans Git, sans commit sur la branche. C'est le point de retour en cas d'escalade. DELIVER en prend d'autres, plus fins :

au début de chaque scénario et après chaque pas validé (le « dernier point sûr »).

3.2 Le dossier de passation

Chaque agent démarre dans une **session neuve**. Il ne voit jamais la conversation d'un autre agent : tout ce qu'il sait vient du dossier de passation, construit par le code. Il contient, dans l'ordre :

1. la story : identifiant, titre, énoncé, critères numérotés ;
2. le niveau de difficulté et le dossier de travail `.skraft/<slug>/` ;
3. la phase en cours, les phases terminées avec leurs livrables, les phases sautées ;
4. les ADR décidés (identifiant, statut, chemin) ;
5. les réponses de l'utilisateur (questions ouvertes, questions bloquantes) ;
6. les tests d'acceptance requalifiés en non-régression, et la mention « gelés » dès qu'une empreinte existe ;
7. en DELIVER : les scénarios livrés et les constats ;
8. le **rejet précédent** : les motifs exacts du dernier refus, quand il y en a un.

Chaque phase y ajoute ce qui lui est propre : présélection de fichiers en RESEARCH, emplacement des tests d'acceptance en DISTILL, scénario et pas en cours en DELIVER. Le classifieur et les lentilles reçoivent un dossier différent, décrit à leur étape.

3.3 Le tour d'un agent

Un tour se termine de trois façons :

- **l'outil terminal** (`report_phase`, `submit_lens_verdict`, `submit_difficulty`) : la sortie est validée par un schéma, puis la porte s'exécute ;
- **une question bloquante** (`report_blocker`) : la tâche passe au stade BLOCKED et la question part à l'humain. Sa réponse est ajoutée au dossier, puis l'agent est relancé **avec le même retour** qu'avant sa question. Aucun essai n'est consommé ;
- **un tour interrompu** (silence, délai dépassé, budget de tokens ou de contexte) : c'est un défaut de travail, traité comme un rejet avec le motif « Le tour précédent a été interrompu (…) : aucune porte n'a jugé ton travail ».

Pendant le tour, chaque écriture passe par des **gardes** : hors du périmètre du rôle, dans un fichier en lecture seule, dans `.git` : refus immédiat, avec le motif donné à l'agent.

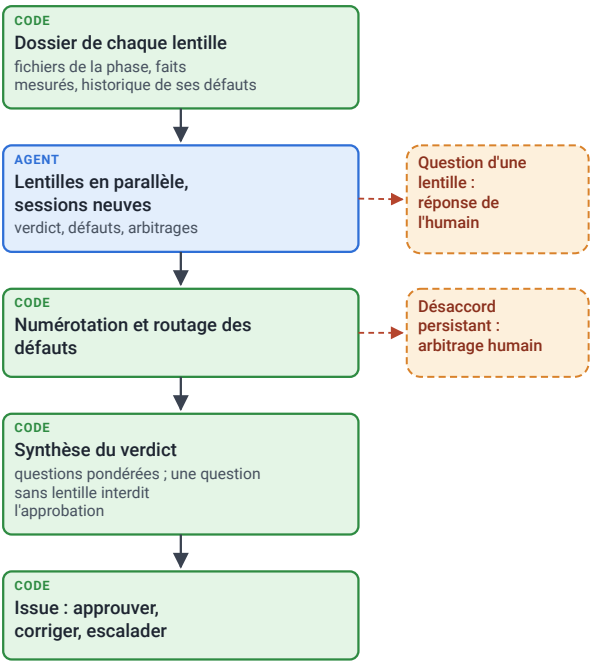
3.4 La porte

La porte est entièrement exécutée par le code. Elle ne fait confiance à aucune déclaration de l'agent : elle relit les fichiers, relance elle-même les tests, compare des empreintes. Elle rend soit « passe », avec les livrables enregistrés (chemin et sha256), soit un **rejet** nommé (par exemple « citations ») avec des motifs exacts.

3.5 La revue par lentilles

DESIGN, DISTILL et DELIVER se terminent par une revue. RESEARCH n'en a pas.

Revue par lentilles



3.5.1 Les quatre lentilles

cold-reader
Angle : Vocabulaire métier, clarté des noms, intention visible. Lit sans aucun contexte du producteur. Gravité plafonnée par le code à `major` : la clarté ne bloque jamais seule.
Ce qu'elle reçoit : Livrables, code, tests. **Aucun fait mesuré.**

test-integrity
Angle : Théâtre de test : test tautologique, assertion qui ne peut pas échouer, test modifié pour passer.
Ce qu'elle reçoit : Critères, tests, code, livrables, faits.

architecture
Angle : Sens des dépendances, aucune doublure du domaine dans les tests, fidélité aux ADR et au modèle.
Ce qu'elle reçoit : ADR acceptés, code, tests, livrables, faits.

quality-gates
Angle : Cohérence des livrables avec les mesures du code ; n'exécute et ne rejuge rien.
Ce qu'elle reçoit : Faits et livrables ; ni code ni tests.

En DESIGN, `quality-gates` n'intervient pas : il n'y a encore rien de mesuré. Au niveau `simple`, seules `cold-reader` et `test-integrity` revoient.

Les **faits** sont établis par le code et présentés comme acquis : livrables et empreintes, résultat de la dernière exécution des tests, séquences rouge puis vert, mutation, couverture, dérogations, réponses de l'utilisateur. Une mesure absente est dite absente, jamais présentée comme favorable.

En DESIGN et en DISTILL, les lentilles jugent des documents, pas du code : leur dossier ajoute des **points à examiner** propres à la phase. Par exemple, en DESIGN, `architecture` vérifie que chaque agrégat garantit ses invariants et que chaque patron coûteux est justifié ; en DISTILL, `test-integrity` vérifie que chaque cas limite tranché par les critères a son scénario. Ces

points viennent des relecteurs de DESIGN et de DISTILL de skraft-plugin, pour la part qui demande un jugement ; le reste est vérifié par les portes. En DELIVER, `test-integrity` vérifie en plus que chaque test respecte la couche et la doublure prévues par la matrice du plan de test.

3.5.2 Ce que rend une lentille

L'outil `submit_lens_verdict` porte :

- le **verdict** : `APPROVED`, `CHANGES_REQUESTED` ou `REJECTED` ;
- les **défauts** : gravité (`blocker`, `major`, `minor`), cible (`test`, `code`, `deliverable`), fichier et ligne si possible, description ;
- les **arbitrages** (`rulings`) sur ses défauts précédents que l'agent a contestés : `accepted` ou `maintained`, avec une raison.

3.5.3 La synthèse, calculée par le code

Les lentilles tournent en parallèle, chacune isolée. Le code calcule ensuite le verdict, dans cet ordre :

1. aucune lentille n'a répondu : **CHANGES_REQUESTED** ;
2. deux lentilles `REJECTED` ou plus : **REJECTED** (une seule ne suffit pas) ;
3. un défaut `blocker` : **CHANGES_REQUESTED** ;
4. une des quatre questions sans lentille pour y répondre : **CHANGES_REQUESTED**. Les questions sont la complétude (poids 0,3), l'adéquation métier (0,3), la qualité (0,15) et le risque (0,25). Une question restée sans réponse interdit l'approbation ;
5. toutes les lentilles `APPROVED` et aucun défaut `major` : **APPROVED** ;
6. sinon : **CHANGES_REQUESTED**.

Les contributions sont publiées question par question dans le rapport de revue (`.skraft/<slug>/reviews/<phase>-review-<n>.md` et `.json`), jamais additionnées en un score unique.

3.5.4 Les fils de défauts

Chaque défaut `blocker` ou `major` devient un **fil**, numéroté par le code (`lentille#tour.n`), avec une cible effective : un défaut qui cite un fichier de test va au testeur, quoi qu'en dise la lentille. L'agent répond à chaque fil dans `report_phase` :

- `fixed` : corrigé ;
- `contested`, avec une raison : la lentille arbitrera à la revue suivante ;
- `outOfScope`, avec une raison : juste, mais pour l'autre agent (renvoyé une seule fois).

open
Sens : signalé, à traiter
Suite : l'agent le reçoit

contested
Sens : contesté par l'agent
Suite : la lentille arbitre

accepted
Sens : contestation acceptée par la lentille
Suite : le défaut tombe

maintained
Sens : contestation refusée
Suite : à corriger ; une nouvelle contestation le rend « recontested »

recontested
Sens : contesté une seconde fois

Suite : si la lentille maintient encore : l'humain tranche

humanAccepted
Sens : l'humain donne raison à l'agent
Suite : le défaut tombe

humanMaintained
Sens : l'humain donne raison à la lentille
Suite : à corriger, sans recours

fixed
Sens : corrigé, ou tenu pour traité en fin de tour
Suite : la lentille vérifie à la revue suivante

3.5.5 L'issue de la revue

- **APPROVED** : commit de la phase ; en DESIGN, ratification des ADR d'abord.
- **REJECTED** : escalade immédiate, avec les lentilles qui rejettent et leurs défauts.
- **CHANGES_REQUESTED** : nouvel essai de la phase avec les défauts en motif. En DELIVER, une **correction ciblée** remplace le réessai (voir Chapitre 8.8). La **première revue** d'une phase a toujours droit à un essai, même si le budget est épuisé par les portes (ADR 0042).

3.5.6 Question ou silence d'une lentille

Une lentille peut poser une question (`report_blocker`) au lieu de deviner une décision métier. La question part à l'humain ; la phase est rejouée avec la réponse, **sans consommer d'essai** (en DELIVER, seule la revue est relancée). Au-delà de deux questions de revue dans une phase : escalade. Une lentille muette, ou qui repose une question déjà tranchée, est relancée seule,

une fois ; au second échec : escalade « revue impossible ».

3.6 Le budget d'essais et le rejeu

Rejet de porte ou de revue, budget restant
Ce que fait le code : relance ciblée du spécialiste avec les motifs
Dépôt : inchangé (pas de rembobinage)
Sessions : gardées : l'agent corrige ce qu'il a écrit

Budget épuisé
Ce que fait le code : escalade : relancer, sauter, abandonner
Dépôt : rembobiné au checkpoint de phase
Sessions : oubliées

Verdict REJECTED
Ce que fait le code : escalade
Dépôt : rembobiné
Sessions : oubliées

Panne d'environnement
Ce que fait le code : escalade immédiate, sans essai consommé
Dépôt : rembobiné
Sessions : oubliées

Question bloquante
Ce que fait le code : réponse de l'humain, puis relance avec le même retour
Dépôt : inchangé
Sessions : gardées

Le rembobinage et l'oubli des sessions vont **toujours ensemble** (ADR 0050) : un agent qui se souviendrait d'avoir écrit des fichiers effacés éditerait des fichiers absents.

Après une escalade, l'humain choisit :

- **relancer** : la phase repart, budget remis à zéro, avec le motif de l'escalade ;
- **sauter** : la phase est ajoutée aux phases sautées ; son motif et ses défauts ne passent pas à la suivante ;
- **abandonner** : la tâche s'arrête (stade ABORTED).

3.7 Le commit et la traçabilité

Une phase approuvée est commitée sur la branche `skraft/<slug>` : `docs(skraft): <phase> de <slug>`, ou `feat(<slug>): livraison de la story` pour DELIVER. Juste avant, le code réécrit les rapports de revue du run en cours (ceux d'un run rembobiné ne ressuscitent pas) et, en DELIVER, le fichier des dérogations. Si le commit échoue (verrou Git, disque plein), rien n'est rembobiné : l'humain choisit de relancer le commit, de continuer sans commiter ou d'abandonner.

La **trace** de la tâche raconte tout : chaque transition, chaque effet, chaque rejet (porte et motifs exacts), chaque arbitrage (qui, quoi, pourquoi). Elle ne fait foi de rien : seules les preuves, les exécutions de tests enregistrées par le code, font foi.

4 Préparation : INIT, CLASSIFY et ROUTE

Ces trois étapes ne produisent aucun livrable. Elles préparent la tâche, fixent les moyens engagés et lèvent les ambiguïtés métier avant de concevoir.

4.1 INIT : lire la story et vérifier le terrain

4.1.1 Rôle

Lire et valider la story, vérifier que les tests existants sont sains, ouvrir la branche de travail.

4.1.2 Entrées et sorties

Entrée
Contenu : Le chemin de la story (par exemple `stories/us-12.md`), donné à la commande `/skraft`.

Format exigé
Contenu : Un titre `# US-12 — Titre` ; une section « Critères d'acceptation » avec des lignes `1. …` (au moins une) ; une section « Questions ouvertes » facultative, en puces.

Sorties
Contenu : La story analysée (critères, valeurs littérales de chaque critère, questions numérotées `q1`, `q2` …) ; la branche `skraft/<slug>` ; les dossiers `.skraft/<slug>/research`, `/design`, `/distill/features`, `/reviews` ; un checkpoint « qualité initiale » qui sert de base à la couverture du diff de toute la tâche.

4.1.3 Qui exécute quoi

Tout est exécuté par le `CODE` ; aucun agent n'intervient.

4.1.4 Le précontrôle

Avant tout appel d'agent, le code lance les tests du projet.

- **Bloquant** : délai dépassé, sortie en erreur, tests en échec ou ignorés. Message « Lancement skraft bloqué avant tout appel d'agent », puis escalade : relancer ou abandonner.
- **Non bloquant** : une dette de couverture sur le code existant. Message « Précontrôle skraft — avertissement non bloquant sur le code existant » : les seuils ne porteront que sur les lignes ajoutées ou modifiées.

4.1.5 Erreurs

Story introuvable ou mal formée, ou précontrôle rouge : escalade « relancer, abandonner ».

4.2 CLASSIFY : estimer la difficulté

4.2.1 Rôle

Choisir le niveau de difficulté, qui fixe les moyens engagés (voir le tableau du chapitre 2).

4.2.2 Qui exécute quoi

`CODE`

Ce qu'il fait : Calcule quatre indicateurs : nombre de critères, modules touchés (dossiers de `src/`, `Sources/` ou `packages/` cités par la story), nouvelle intégration externe (API, SQL, Kafka… nommés mais absents du dépôt), nombre de questions ouvertes.

Applique une grille : `challenging` si une intégration nouvelle touche 3 modules ou plus, ou s'il y a 2 questions ou plus ; `medium-hard` si plus de 5 critères, 3 modules ou plus, ou une question ; `medium` si 3 critères ou plus ou 2 modules ; `simple` sinon.

AGENT `skraft-classifier`
Ce qu'il fait : Reçoit seulement la story, les indicateurs et le niveau de la grille (pas le dossier de passation). Rend `submit_difficulty` : un niveau et une justification.

HUMAIN
Ce qu'il fait : Confirme le niveau, ou choisit un cran au-dessus ou en dessous. En mode `--auto`, le niveau proposé est retenu sans question.

4.2.3 Premier passage et rejeu

Un niveau inconnu, un silence ou un délai dépassé relancent le classifieur ; chaque relance est tracée comme un rejet de la porte « classifieur ». Au-delà du budget : escalade « relancer, abandonner ». La difficulté n'est écrite qu'une fois : elle ne change plus ensuite.

4.3 ROUTE : trancher les questions ouvertes

4.3.1 Rôle

Obtenir de l'humain une réponse à chaque question métier avant de concevoir.

4.3.2 Entrées et sorties

Entrées

Contenu : Les questions de la story, et celles que la recherche a ajoutées (section « Passage de relais » de `research.md`).

Sorties
Contenu : Une réponse par question : le texte de l'humain, « (réponse vide) », ou « (question reportée par l'utilisateur) ». Les réponses entrent dans tous les dossiers de passation et dans les faits donnés aux lentilles.

4.3.3 Qui exécute quoi

Le CODE pose chaque question une seule fois ; l'HUMAIN répond ou reporte. ROUTE n'existe que s'il reste des questions, et seulement avant une phase autre que RESEARCH.

4.3.4 Erreurs

Si la création d'une demande échoue (panne), escalade « relancer, abandonner » : relancer repose les seules questions dont la demande n'a pas été créée. Une réponse donnée pendant l'escalade est gardée.

5 RESEARCH : comprendre l'existant

5.1 Rôle

Explorer le dépôt pour savoir où et comment la story s'insère, évaluer des approches, en recommander une, et faire remonter les questions métier. Sautée aux niveaux `simple` et `medium`.

5.2 Entrées et sorties

Entrées
Contenu : Le dossier de passation, plus une **présélection** d'au plus 20 fichiers : le code les choisit d'après les mots de la story (4 lettres ou plus, hors mots vides) présents dans leur chemin.

Sortie de l'agent
Contenu : `.skraft/<slug>/research/research.md`, avec 8 sections : Périmètre et critères de succès ; Fichiers analysés ; Recherches effectuées ; Conventions du projet ; Découvertes ; Approches évaluées ; Recommandation ; Passage de relais.

Sorties vers la suite
Contenu : Le document (livrable, avec son sha256) et de nouvelles questions ouvertes, extraites de la section « Passage de relais » : sous-section « Questions ouvertes » si elle existe, sinon les lignes terminées par « ? ». Elles seront posées en ROUTE.

5.3 Qui exécute quoi

AGENT `skraft-researcher`
Ce qu'il fait : Lit le dépôt (`list_files`), écrit `research.md`, peut poser une question bloquante. Il n'a aucun outil web.

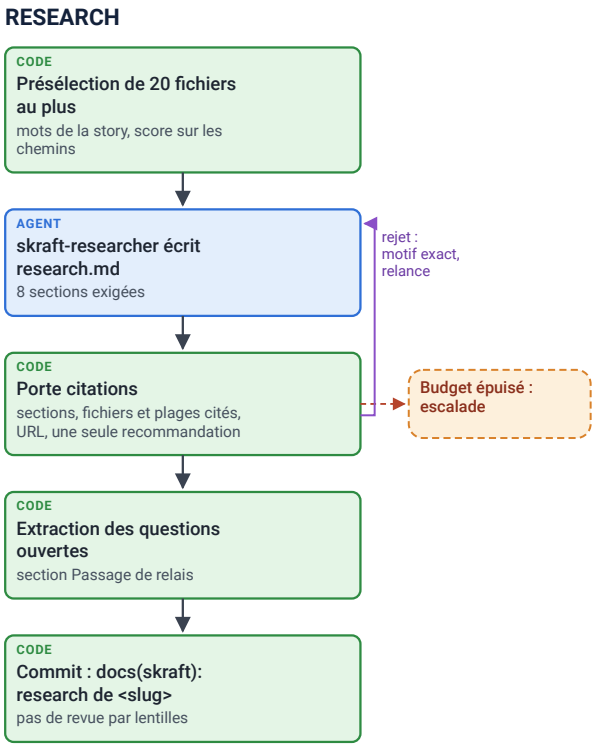
CODE
Ce qu'il fait : Présélection des fichiers ; porte « citations » ; extraction des questions ; commit `docs(skraft): research de <slug>`. Pas de revue par lentilles.

5.4 La porte « citations »

Entièrement exécutée par le code. Motifs de rejet :

- document absent : « Document de recherche absent » ;
- section manquante : « Section manquante : « … ». » ;
- chemin cité (`fichier:début-fin`) : « fichier inexistant dans le dépôt », « plage inversée », « plage hors du fichier, qui compte N lignes » ;
- URL mal formée (http ou https, hôte avec un point, ou `localhost`) ;
- recommandation absente, ou plus d'une recommandation.

5.5 Organigramme



5.6 Premier passage et rejeu

Au premier passage, l'agent part du dossier et de la présélection. Après un rejet de la porte, il est relancé **dans la même session**, avec les motifs exacts ; ses fichiers restent sur le disque, il les corrige. Budget épuisé : escalade, rembobinage de la phase et oubli de la

session. Si l’humain relance, l’agent repart d’une session neuve avec le motif de l’escalade.

6 DESIGN : concevoir et décider

6.1 Rôle

Modéliser le domaine touché par la story, fixer les contrats, et rédiger les décisions d'architecture (ADR), que l'architecte humain ratifie.

6.2 Entrées et sorties

Entrées
Contenu : Le dossier de passation : story, réponses aux questions, `research.md` s'il existe, ADR déjà décidés.

Sorties
Contenu : Dans `.skraft/<slug>/design/` : `event-model.md`, `domain-model.md`, `context-map.md` (modélisateur) ; `contracts.md`, `diagrams.md` (décideur) ; `consistency-matrix.md` (écrite par le code). Dans `docs/adr/` : les ADR, au statut `Proposed` puis `Accepted` ou `Rejected`.

Vers la suite
Contenu : Les ADR décidés (cités dans chaque dossier de passation, en lecture seule pour tous les agents) ; les noms du contrat (`contracts.md`), que DELIVER protège pendant une correction.

6.3 Qui exécute quoi

AGENT `skraft-architect-model`
Ce qu'il fait : Premier tour : modèle d'événements, modèle du domaine, carte des contextes. Les noms sont en PascalCase entre accents graves.

AGENT `skraft-architect-decide`
Ce qu'il fait : Second tour, avec le même motif : contrats et diagrammes, en ne citant que des noms définis par le modélisateur, chacun avec la même catégorie partout. Écrit les ADR **uniquement** par l'outil `render_adr` ; il n'a pas d'accès direct à `docs/adr/`.

CODE `render_adr`
Ce qu'il fait : Valide l'ADR (champs non vides, statut `Proposed`), le numérote (plus grand numéro + 1), l'écrit avec le gabarit (Contexte, Décision, Conséquences, Alternatives). Un même titre encore proposé est réécrit, pas dupliqué. Refuse d'amender un ADR déjà décidé.

CODE **porte « cohérence »**
Ce qu'il fait : Voir ci-dessous.

AGENT **lentilles**
Ce qu'il fait : Revue par `cold-reader`, `test-integrity` et `architecture` (pas `quality-gates`).

HUMAIN **architecte**
Ce qu'il fait : Ratifie chaque ADR proposé.

6.4 La porte « cohérence »

- `event-model.md` doit exister. C'est le seul document obligatoire.
- Tout nom en PascalCase cité seul entre accents graves dans `contracts.md` ou `diagrams.md` doit apparaître dans `event-model.md` ou `domain-model.md`.
- Un ADR existant ne se modifie pas : on en rédige un nouveau qui le remplace (append-only).

- Chaque nouvel ADR : un titre, toutes les sections, le statut `Proposed`, pas de verdict dans le nom de fichier, pas de titre de bonne pratique (« ne pas », « toujours »…).
- Numérotation sans doublon ni trou ; registre des remplacements (`supersessions.md`) cohérent dans les deux sens.
- Chaque commande de `event-model.md` (« Commande `X` ») a au moins un événement : « `X` → événement `Y` », ou une ligne de tableau qui les associe. Une requête en est exemptée.
- Chaque relation entre contextes de `context-map.md` (« `A` → `B` ») nomme son patron sur la même ligne : ACL, Conformist, Shared Kernel, Customer-Supplier, Open Host Service, Published Language, Partnership ou Separate Ways.
- **Matrice de cohérence**, construite par le code : pour chaque nom, la catégorie que lui donne chaque document (« Commande `X` », « objet-valeur `Y` »…), modèle, contrats, diagrammes et nouveaux ADR. Un même nom classé commande et requête, entité et objet-valeur, ou agrégat et service de domaine est refusé. La porte écrit `consistency-matrix.md` à chaque essai.
- Un concept classé dans un nouvel ADR (« Agrégat `X` »…), hors de sa section « Alternatives », existe dans `event-model.md` ou `domain-model.md` . L'inverse, un agrégat qu'aucun ADR ne justifie, est examiné par la lentille `architecture` .
- Chaque contexte de `context-map.md` est classé par une ligne « `A` : Core », « Supporting » ou « Generic ».
- `contracts.md` porte une section par commande, requête et événement du modèle (« `## Commande X`

»…), et chaque dépôt déclare sa couche : « Couche : Domaine » ou « Couche : Application », jamais l'infrastructure.
- Un engagement structurel déjà présent dans les sources du projet (bus CQRS, event store, saga), ou une couche anticorruption de la carte, doit être ratifié par un ADR encore valable, nouveau ou de rattrapage.

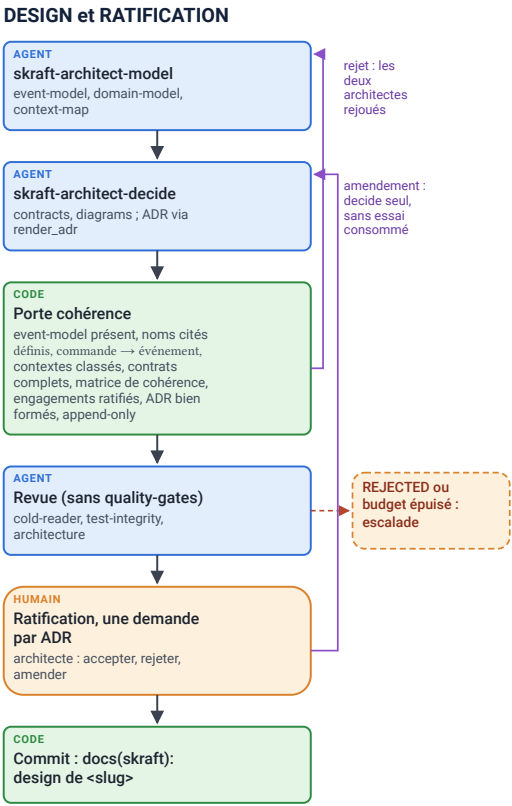
6.5 La ratification

Après une revue approuvée, le code envoie une demande par ADR proposé, réservée à l'architecte :

- **accepter** ou **rejeter** : le statut est écrit aussitôt dans le fichier de l'ADR, qui devient en lecture seule ;
- **amender**, avec un commentaire : le décideur est relancé avec « Amendement demandé sur ADR-N : … », **sans consommer d'essai**. Le résultat repasse par la porte et la revue, puis revient en ratification.

Tant qu'un ADR est `Proposed`, DISTILL est inatteignable.

6.6 Organigramme



6.7 Premier passage et rejeu

Au premier passage : modélisateur, puis décideur, puis porte. Un rejet de la porte ou une revue « changements demandés » rejoue **les deux** architectes, dans leurs sessions, avec le motif. Un amendement ne re-

joue que le décideur. À l'escalade, le dépôt revient au début de DESIGN : les ADR écrits pendant la phase disparaissent du disque et de la liste, même ceux déjà ratifiés ; seuls restent ceux d'avant la phase.

7 DISTILL : écrire les scénarios et les tests d'acceptance

7.1 Rôle

Traduire chaque critère d'acceptation en scénario Gherkin et en test d'acceptance exécutable, rouge tant que la fonctionnalité n'existe pas, puis **geler** ces tests : DELIVER devra les faire passer sans jamais les modifier.

7.2 Entrées et sorties

Entrées
Contenu : Le dossier de passation (story, ADR décidés, livrables de DESIGN), plus l'emplacement des tests d'acceptance de la stack du projet.

Sorties de l'agent
Contenu : Dans `.skraft/<slug>/distill/` : des fichiers `.feature` (chaque scénario tagué `@ac-N`), `test-plan.md`, `impl-plan.md`. Dans les tests d'acceptance du projet : un test par critère, dont le **nom** porte `ac-N`. Éventuellement des bouchons dans les sources.

Sorties du code
Contenu : L'**empreinte** des tests d'acceptance ; la liste des tests requalifiés en non-régression ; les livrables enregistrés avec leur sha256.

Vers DELIVER
Contenu : Les tests gelés, l'empreinte, `impl-plan.md` (ordre des scénarios), les non-régressions.

7.3 Qui exécute quoi

AGENT `skraft-acceptance`
Ce qu'il fait : Écrit les scénarios, les plans et les tests, en recopiant les valeurs des critères caractère par caractère. Peut lancer `run_tests` pour se vérifier, mais cette exécution ne fait pas foi. Pour un test déjà vert (comportement existant), écrit dans `test-plan.md` la ligne `- @ac-N : role: regression — justification : …`.

CODE **quatre portes**
Ce qu'il fait : Dans l'ordre, voir ci-dessous. Le code lance lui-même la suite de tests.

CODE **gel**
Ce qu'il fait : Prend l'empreinte des tests d'acceptance quand les portes passent.

AGENT **lentilles**
Ce qu'il fait : Revue de la phase.

7.4 Les quatre portes, dans l'ordre

gherkin
Ce que vérifie le code : Pas de détail technique dans les étapes (verbe HTTP, chemin d'URL, identifiant technique, base de données, DTO…) ; une seule action `Quand` ; pas d'assertion vague ; 7 étapes au plus ; pas de tag technique (`@api`, `@ui` …).
Motifs (texte exact) : « Le Gherkin décrit ce que fait le métier, pas comment le code s'y prend. » ; « N étapes « Quand ». Un seul déclencheur par scénario » ; « l'assertion « … » ne nomme aucun résultat métier observable »

valeurs
Ce que vérifie le code : Pour chaque critère : un scénario `@ac-N`, un test portant `ac-N`, et chaque valeur du critère présente dans le scénario et dans le test (`12,5` vaut `12.5`).
Motifs (texte exact) : « Aucun scénario tagué @ac-N » ; « La valeur « … » du critère N n'apparaît pas dans le scénario @ac-N »

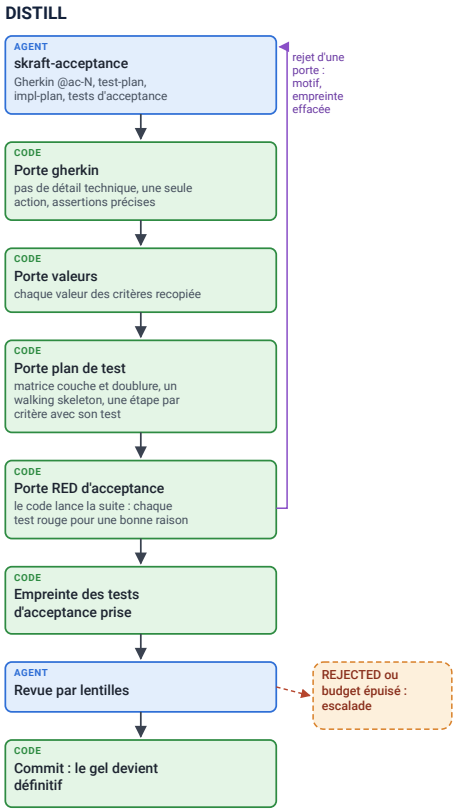
plan de test
Ce que vérifie le code : `test-plan.md` et `impl-plan.md` existent. `test-plan.md` porte une matrice de couverture : une ligne par critère avec sa couche (Domaine, Application, Infrastructure) et sa doublure, et au moins un walking skeleton. `impl-plan.md` cite chaque critère, et chaque critère a une étape qui nomme son fichier de test. Vérifié avant toute exécution des tests.
Motifs (texte exact) : « Plan de test absent : … », « test-plan.md n'a pas de matrice de couverture », « impl-plan.md ne cite pas le critère N », « aucune étape du critère N ne nomme son fichier de test »

RED d'acceptance
Ce que vérifie le code : Le code lance la suite. Chaque test d'acceptance doit échouer pour une bonne raison (assertion ou symbole manquant), ni ignoré, ni en erreur d'une autre nature. Un test déjà vert n'est accepté que s'il est requalifié en non-régression avec sa justification.
Motifs (texte exact) : « … échoue pour une mauvaise raison » ; « … est ignoré » ; « … passe déjà : requalifiez-le en non-régression avec une justification dans test-plan.md »

7.5 Le gel des tests d'acceptance

- L'empreinte est prise quand les portes passent, **avant** la revue. Tant que DISTILL n'est pas commitée, le gel est provisoire : un rejet ultérieur l'efface, et la revue peut encore faire corriger un test. Les lentilles lisent « le gel devient définitif au commit de DISTILL ».
- Au commit de DISTILL, le gel devient définitif. En DELIVER, chaque pas et chaque porte comparent l'empreinte : un test d'acceptance modifié fait annuler le tour ; au début de DELIVER, c'est une panne d'environnement (escalade).
- Si DISTILL a été sautée, rien n'est gelé : DELIVER le signale à l'utilisateur et aux lentilles.

7.6 Organigramme



7.7 Premier passage et rejeu

Après un rejet de porte ou de revue, l'empreinte est effacée et l'agent est relancé avec les motifs, dans sa session. L'empreinte sera reprise au prochain passage des portes. À l'escalade : rembobinage de DISTILL,

empreinte et non-régressions remises à zéro, session oubliée.

8 DELIVER : livrer en TDD, scénario par scénario

8.1 Rôle

Écrire le code qui fait passer les tests d'acceptance gelés, un scénario à la fois, en TDD strict : un test unitaire rouge, le code minimal qui le fait passer, un nettoyage, et recommencer.

8.2 Entrées et sorties

Entrées
Contenu : Les tests d'acceptance gelés et leur empreinte ; `impl-plan.md` ; les non-régressions ; les ADR décidés et le contrat (`contracts.md`) ; le dossier de passation.

Sorties
Contenu : Le code de production et les tests unitaires, commités pas à pas (`test(...)` , `feat(...)` , `refactor(...)` , `fix(...)`), puis le commit `feat(<slug>): livraison de la story` ; `.skraft/<slug>/deliver/derogations.md` s'il y a des dérogations ; le rapport final.

8.3 Qui exécute quoi

AGENT `skraft-test`
Ce qu'il fait : Pas « test » : écrit **un** nouveau test unitaire rouge, ou désigne le test d'acceptance du scénario (`coveredBy`) s'il suffit. Pas « fix-test » des corrections. N'écrit que dans les tests unitaires.

AGENT `skraft-dev`
Ce qu'il fait : Pas « vert » (code minimal), « net-toyage », ou « dev » (les deux en un tour) ; pas « fix » des corrections. N'écrit que dans les sources ; tous les tests sont en lecture seule pour lui.

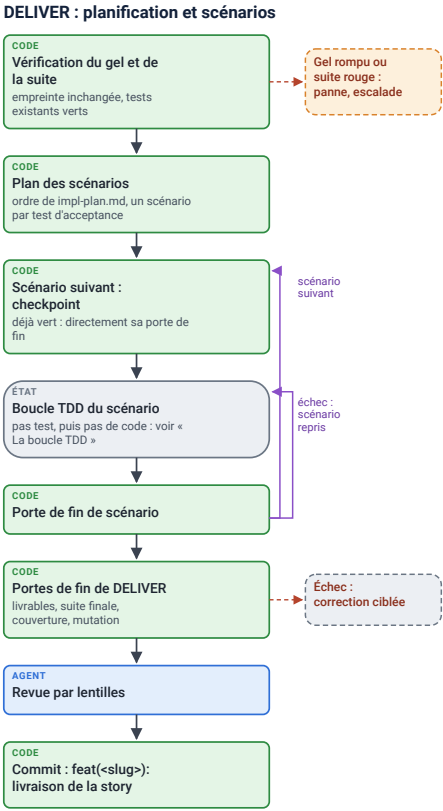
AGENT `skraft-mutation-arbiter`
Ce qu'il fait : Juge les mutants que le testeur conteste comme équivalents.

AGENT **lentilles**
Ce qu'il fait : Revue de la livraison, avec les faits mesurés.

CODE
Ce qu'il fait : Plan, référence des tests, toutes les exécutions de tests qui font foi, toutes les portes, empreintes, mutation, couverture, linter, règles de dépendance, commits, checkpoints, retours en arrière, routage des défauts, budgets, dérogations.

HUMAIN
Ce qu'il fait : Questions bloquantes, escalades, désaccords persistants, mutants refusés, couverture contestée.

8.4 Planification

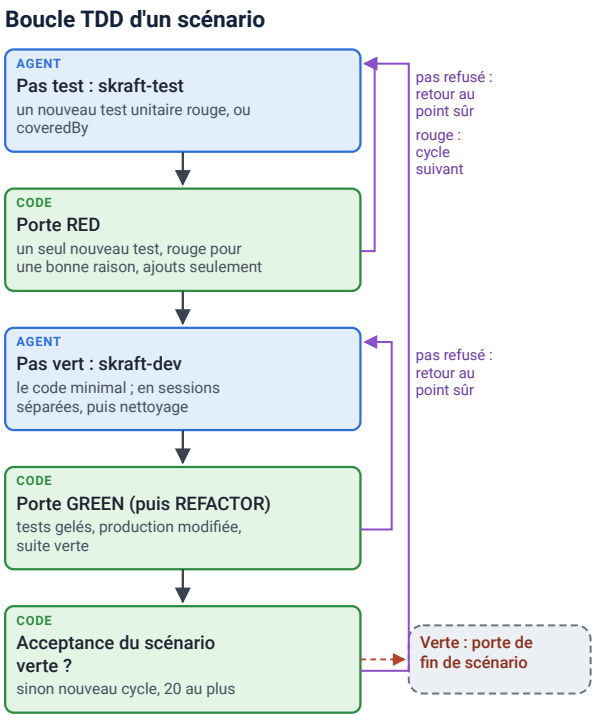


Avant tout agent, le code :

1. vérifie que les tests d'acceptance n'ont pas changé depuis DISTILL (sinon panne : « Les tests d'acceptance ont changé depuis DISTILL alors

qu'ils sont gelés : restaurez-les avant de relancer. ») ;
2. lance la suite : tout test hors acceptance doit être vert (sinon panne : « La suite est rouge avant DELIVER : … ») ; le résultat devient la **référence** des tests ;
3. planifie un scénario par test d'acceptance, dans l'ordre où `impl-plan.md` cite les critères, puis les autres par numéro. Un test requalifié en non-régression est un scénario déjà fait.

8.5 La boucle TDD d'un scénario



La granularité dépend du niveau. En **un tour par scénario**, le développeur fait passer le test au vert puis nettoie dans le même tour. En **sessions séparées** (niveau `challenging`), les pas « vert » et « nettoyage » sont deux tours distincts. Un scénario qui échoue en mode « un tour » est repris en sessions séparées, et les scénarios suivants le restent.

test
Agent : `skraft-test`
Porte du code : ce qu'elle exige : Tests d'acceptance intacts ; seuls des tests unitaires modifiés ; un test existant ne reçoit que des ajouts ; **un seul** nouveau test, rouge pour une bonne raison (ni déjà vert, ni ignoré, ni en erreur d'une autre nature) ; aucun test vert devenu rouge. Ou bien `coveredBy` : aucun fichier modifié, et le test d'acceptance du scénario rouge pour une bonne raison.

vert / dev
Agent : `skraft-dev`
Porte du code : ce qu'elle exige : Tests unitaires inchangés depuis leur rouge (par empreinte) ; au moins un fichier de production modifié ; suite verte, sauf les tests d'acceptance des scénarios en attente ; le test du pas passe.

nettoyage
Agent : `skraft-dev`
Porte du code : ce qu'elle exige : Les mêmes contrôles, sans exiger de modification ; et l'acceptance du scénario, verte avant, le reste. Un nettoyage refusé est annulé (retour au vert), sans escalade.

Après chaque pas validé : commit, puis checkpoint (le nouveau « dernier point sûr »). Si l'acceptance du scénario est verte, on passe à sa porte de fin ; sinon, un nouveau cycle commence, 20 au plus.

8.6 La porte de fin de scénario

Le code relance la suite et cumule les motifs :

- tests d'acceptance intacts ; acceptance du scénario verte ; suite verte hors acceptance des autres scénarios en attente ;
- pour chaque test unitaire du scénario, une exécution rouge (assertion ou symbole manquant) **puis** une verte, dans les preuves enregistrées par le code ;
- sur les fichiers de production modifiés pendant le scénario : couverture du diff, linter du projet, règles de dépendance (par défaut, le domaine n'importe pas l'infrastructure) ; mutation, si la barre de qualité la demande à ce moment-là.

Une mesure impossible (pas de couverture, pas de linter) devient une **note** pour les lentilles, jamais un succès.

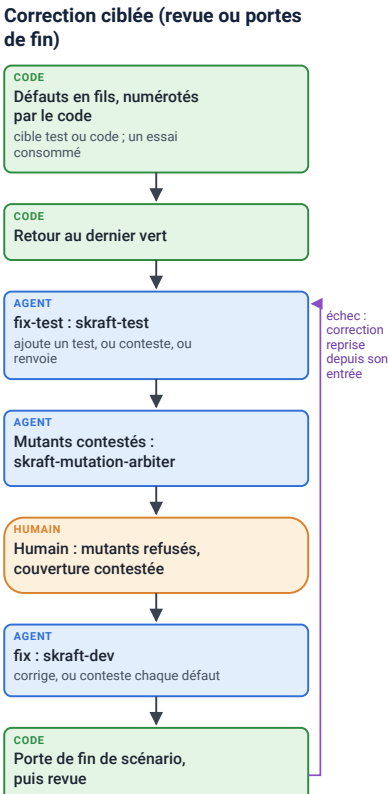
8.7 Les portes de fin de DELIVER

Quand tous les scénarios sont faits, le code vérifie l'ensemble de la tâche :

1. **livrables** : au moins un fichier modifié pendant la phase (sinon escalade, sans essai consommé) ;
2. **suite finale** : la suite complète, verte ;
3. **couverture du diff** : sur toutes les lignes ajoutées ou modifiées depuis le début de la tâche, moins celles exemptées par dérogation ;
4. **mutation** (par défaut à ce moment-là) : seuils par couche (100 % pour le domaine), moins les mutants exemptés par dérogation.

Un échec de ces portes ne rejoue pas la phase, où tout est déjà vert : il ouvre une **correction ciblée**, adressée à l'agent concerné.

8.8 Corrections ciblées et arbitrages



Une correction ciblée (revue « changements demandés » ou porte de fin en échec) consomme un essai et repart du dernier vert.

- Chaque défaut devient un fil numéroté : un par mutant survivant, un par fichier mal couvert, un par dé-

faut de revue. Les défauts visant un test vont au testeur (pas « fix-test »), les autres au développeur (pas « fix »).
- L'agent corrige, conteste avec une raison, ou renvoie à l'autre agent (une fois). Un développeur qui conteste ou renvoie **chaque** défaut peut ne modifier aucun fichier.
- Les tests ajoutés par le testeur et encore rouges deviennent les cibles du développeur. Pendant un pas « fix », le code vérifie aussi que les noms du contrat ratifié présents au moment de la revue restent dans le code.

Mutant contesté (ADR 0065) :

1. l'AGENT `skraft-mutation-arbiter`, distinct du testeur, juge chaque mutant : équivalent (aucune entrée ne le distingue du code) ou non ;
2. équivalent : **dérogation accordée par l'agent** ; le mutant sort du score ; au plus deux par livraison ;
3. non équivalent, ou plafond atteint : l'HUMAIN tranche, avec la contestation et l'avis de l'arbitre sous les yeux : dérogation, ou « à corriger » sans recours.

Couverture contestée (fin de DELIVER) : l'HUMAIN tranche directement : dérogation sur les lignes précises, ou « à tester » sans recours.

Chaque décision est tracée dans la trace de la tâche ; les dérogations sont commitées dans `derogations.md`, montrées aux lentilles et rappelées dans le rapport final.

8.9 La revue de DELIVER

Les lentilles reçoivent les faits de la livraison : tests d'acceptance gelés et vérifiés à chaque pas, pour chaque scénario son test d'acceptance vert et ses tests unitaires « rouges puis verts dans les preuves », mutation, couverture, dérogations, notes. Une revue « changements demandés » déclenche une correction ciblée tant que le budget le permet. Si elle ne relève que des défauts mineurs, ils sont corrigés comme des majeurs ; si elle ne relève aucun défaut, l'humain décide, sans essai consommé.

8.10 Premier passage et rejeu

Pas refusé
Ce que fait le code : Retour au dernier point sûr, relance du même agent avec le motif ; au budget du pas épuisé, le scénario est repris depuis son début.

Scénario en échec
Ce que fait le code : Retour au début du scénario, avec la référence de tests et l'empreinte de ce moment ; relance avec les motifs ; au budget épuisé, escalade (rembobinage du scénario et oubli des sessions du testeur et du développeur).

Correction en échec
Ce que fait le code : Retour à l'entrée de la correction, avec ses défauts d'entrée et sa mémoire (empreinte, référence) ; reprise avec la cause de l'échec.

Panne pendant un scénario ou une correction

Ce que fait le code : Escalade immédiate ; « relancer » reprend ce scénario, ou la correction elle-même, sans rien rejouer de livré.

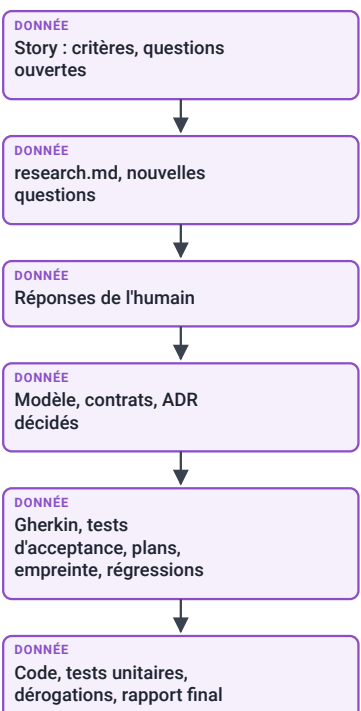
Réessai de la phase (revue sans correction ciblée possible)
Ce que fait le code : Pas de rembobinage : nouveau plan, les scénarios déjà verts passent directement à leur porte de fin.

Relancer après une escalade de toute la phase
Ce que fait le code : Le dépôt est revenu au début de DELIVER : nouveau plan, tous les scénarios sont refaits.

9 Interactions entre les étapes

Les étapes ne se parlent jamais directement : chacune écrit des fichiers sur le disque et des informations dans l'état de la tâche, et le code les transmet à la suivante par le dossier de passation, les portes et les faits de revue.

Données transmises d'une étape à l'autre



Story : critères, valeurs, questions
Produite par : l'auteur (fichier `stories/…`) ; analysée par INIT
Consommée par : toutes les étapes
Comment : dossier de passation ; porte « valeurs » de DISTILL

Niveau de difficulté
Produite par : CLASSIFY (agent et humain)
Consommée par : toutes
Comment : budget, lentilles, RESEARCH sautée ou non, granularité de DELIVER

`research.md`
Produite par : RESEARCH (agent)
Consommée par : DESIGN et suivantes
Comment : livrable cité dans le dossier de passation

Questions ouvertes
Produite par : story ; RESEARCH (section « Passage de relais »)
Consommée par : ROUTE (humain)
Comment : une demande par question

Réponses de l'humain
Produite par : ROUTE ; questions bloquantes ; questions de revue
Consommée par : tous les agents et lentilles suivants
Comment : dossier de passation ; faits de revue. Gardées même après un rembobinage.

Modèle, contrats, matrice
Produite par : DESIGN (agents ; matrice par le code)
Consommée par : DISTILL, DELIVER, lentilles
Comment : livrables ; noms du contrat protégés pendant les corrections de DELIVER

ADR décidés
Produite par : DESIGN (agent, puis humain)
Consommée par : toutes les étapes suivantes
Comment : dossier de passation ; lecture seule ; lentille architecture

Scénarios Gherkin, `test-plan.md`
Produite par : DISTILL (agent)
Consommée par : DELIVER
Comment : le Gherkin du scénario est joint à chaque pas

`impl-plan.md`
Produite par : DISTILL (agent)
Consommée par : DELIVER
Comment : ordre des scénarios

Tests d'acceptance et empreinte
Produite par : DISTILL (agent, puis code)
Consommée par : DELIVER
Comment : gelés : vérifiés à chaque pas et à chaque porte

Non-régressions
Produite par : DISTILL (code, d'après `test-plan.md`)
Consommée par : DELIVER
Comment : scénarios déjà faits

Code, tests unitaires
Produite par : DELIVER (agents)
Consommée par : portes de DELIVER, lentilles, rapport final
Comment : commits pas à pas

Dérogations
Produite par : DELIVER (arbitre ou humain)
Consommée par : portes de mutation et de couverture, lentilles, rapport final

Comment : `derogations.md` ; trace

Rapports de revue
Produite par : chaque revue (code)
Consommée par : l'humain, la trace
Comment : `.skraft/<slug>/reviews/`, commités avec la phase

Motif du dernier rejet
Produite par : toute porte, toute revue
Consommée par : l'agent relancé
Comment : « Rejet précédent » du dossier ; pas transmis à la phase suivante

9.1 Ce qui ne passe pas d'une étape à l'autre

- Les **conversations** des agents : chaque rôle démarre en session neuve à chaque phase.
- Le **motif** d'une phase sautée ou commitée : la phase suivante part sans « Rejet précédent ».
- Les **défauts de revue** d'une phase : ils sont vidés au commit et au saut.
- Les **dérogations** d'un DELIVER rembobiné : le code effacé emporte les exemptions (la trace les garde).

10 Erreurs : bloquantes et non bloquantes

10.1 Les catégories d'erreur

Défaut de travail
Origine : Un tour d'agent sans sortie, un délai, un budget de tokens ou de contexte dépassé.
Traitement : Compté comme un rejet : relance avec « Le tour précédent a été interrompu (…) : aucune porte n'a jugé ton travail. » Escalade au budget épuisé.

Rejet de porte ou de revue
Origine : Une vérification du code a échoué.
Traitement : Relance ciblée avec les motifs, dans le budget ; escalade ensuite.

Panne d'environnement
Origine : Ce qui ne dépend pas de l'agent : outil absent, tests impossibles à lancer, Git, barre de qualité invalide, tests gelés modifiés au début de DELIVER.
Traitement : Escalade immédiate, sans essai consommé.

Erreur inattendue
Origine : Toute autre exception d'un effet.
Traitement : Traitée comme une panne : « Erreur inattendue : … ». Jamais de réessai aveugle.

Question bloquante
Origine : Un agent ou une lentille manque d'une décision métier.
Traitement : La tâche attend l'humain ; aucun essai consommé.

Violation d'une garde
Origine : Écriture hors périmètre, dans un fichier gelé, dans `.git` ; shell.
Traitement : L'outil est refusé, le motif est donné à l'agent, qui continue son tour.

10.2 Les situations bloquantes

La tâche attend une décision humaine

- Confirmation du niveau de difficulté.
- Questions ouvertes (ROUTE), questions bloquantes d'un agent ou d'une lentille.
- Ratification d'un ADR.
- Désaccord persistant sur un défaut, mutant refusé par l'arbitre, couverture contestée.
- Toute escalade (voir ci-dessous).

Escalade : la main passe à l'humain

- Story introuvable ou mal formée ; précontrôle rouge (tests existants en échec, ignorés, en erreur ou trop lents).
- Budget d'essais épuisé dans une phase ou un scénario.
- Verdict de revue `REJECTED`.
- « revue impossible » (lentille muette deux fois) ; plus de deux questions de revue dans une phase.
- Revue « changements demandés » sans aucun défaut à corriger ; DELIVER sans aucun livrable.
- Panne d'environnement, par exemple :

  - « Les tests d'acceptance ont changé depuis DISTILL alors qu'ils sont gelés : restaurez-les avant de relancer. » ;
  - « La suite est rouge avant DELIVER : … » ;
  - « Les tests ont dépassé le délai avant le début de DELIVER. » ;
  - une barre de qualité illisible.
- Commit de phase impossible (verrou Git, disque plein).

INIT, CLASSIFY
Options : relancer, abandonner

ROUTE (questions)
Options : relancer (repose les questions non posées), abandonner

une phase
Options : relancer la phase, sauter la phase, abandonner

un scénario de DELIVER
Options : relancer (ce scénario, ou la correction en cours), sauter la phase, abandonner

le commit d'une phase
Options : relancer le commit, continuer sans commiter, abandonner

10.3 Les situations non bloquantes

Elles sont **signalées**, jamais transformées en succès : un message dans le fil de la tâche, et une note « À noter » dans les faits des lentilles, qui doivent en tenir compte.

Avertissements et notes

- « Précontrôle skraft — avertissement non bloquant sur le code existant » : dette de couverture initiale, ou « Couverture initiale non mesurée ».
- Barre de qualité lue à l'ancien emplacement `.skraft/quality.json` ; couche du cœur qui ne correspond à aucun fichier.
- « Mutation non mesurée : la stack du projet ne propose pas d'outil de mutation. » ; « Mutation désactivée par la barre de qualité du projet. »
- « Couverture non mesurée : la commande de test du projet n'en produit pas. »
- « Qualité de forme non mesurée : le projet n'a pas de linter configuré pour cette stack. »
- « Règle de dépendance non vérifiée : … »
- « DISTILL a été sautée : les tests d'acceptance ne sont pas gelés. DELIVER ne peut pas vérifier qu'ils restent inchangés. »
- Nettoyage refusé : « Scénario ac-N : nettoyage annulé (…) » ; retour au vert, sans escalade.
- Scénario repris en sessions séparées après un échec.

Sans barre de qualité, les seuils par défaut s'appliquent sans avertissement.

10.4 Les gardes de sécurité

- Les gardes s'exécutent en série, celles du cœur d'abord ; le premier refus l'emporte. Une garde qui plante ou dépasse son délai **refuse** par précaution.

- Écriture hors du périmètre du rôle : refusée. Fichier en lecture seule (barre de qualité, ADR décidés, tests gelés selon le pas) : refusée. `.git` : refusée. Shell : toujours refusé.
- Une écriture qui contourne les outils est rattrapée par **empreinte** : en DELIVER, des tests d'acceptance ou unitaires modifiés font annuler le tour (« Le tour est annulé ») ; au début de DELIVER, c'est une panne d'environnement.

11 Extension et workflow : différences

Les deux implémentations partagent les agents, les portes, les règles et les messages. Les écarts suivants ont été relevés dans le code à la date de ce document.

Forme
Extension : Machine à états : réducteur pur, effets, stades.
Workflow : Script rejouable : appels journalisés, reprise par rejeu.

ADR
Extension : Écrits par l'outil `render_adr` ; le décideur n'a pas accès à `docs/adr/`.
Workflow : Écrits directement par le décideur, avec un gabarit ; une porte « adr » ajoute la ligne `Story` et annule toute modification d'un ADR décidé ou d'une autre story.

Ratification
Extension : Une demande par ADR après la revue, hors de l'essai.
Workflow : Dans l'essai, toutes les demandes à la fois.

Questions ouvertes
Extension : Posées à chaque passage en ROUTE.
Workflow : Posées une fois, toutes ensemble, avant DESIGN.

Précontrôle

Extension : Avant la création de la branche ; échec : escalade relancer ou abandonner.
Workflow : Après la branche ; échec : la tâche se termine en échec.

Oubli des sessions
Extension : À chaque escalade (rembobinage de phase ou de scénario).
Workflow : À chaque rembobinage, y compris un simple pas refusé.

Arbitrage, file humaine
Extension : Couverture contestée avant les mutants.
Workflow : Mutants d'abord, puis couverture.

Rapport final
Extension : Avec revues demandant des changements, réessais par phase, identifiant de suivi.
Workflow : Sans ces trois lignes.

11.1 Écarts relevés en préparant ce document

Ces points ont été vus en relisant le code ; ils ne sont pas encore corrigés.

- **Extension** : le stade `VERDICT_GATE` est déclaré mais jamais atteint (écart S3 de l'audit).
- **Workflow** : un défaut de travail de l'arbitre de mutation n'est pas rattrapé localement ; dans l'extension, il laisse le mutant à l'humain.
- **Workflow** : la mutation par scénario (si la barre la demande) ne tient pas compte des dérogations déjà accordées.

11.2 Sources

Extension : `extensions/skraft/src/` (`reducer.ts` , `deliver-machine.ts` , `effects.ts` , `deliver-effects.ts` , `review*.ts` , `gherkin.ts` , `distill.ts` , `design.ts` , `research.ts`). Workflow : `workflows/skraft/` . Décisions : `docs/adr/` (notamment 0040, 0042, 0046, 0047, 0049, 0050, 0062 à 0066) et l'audit `docs/12-audit-portes-lentilles-skraft.md` .